

Class 10: API Authorization & Data Posting

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

Next Auth

- Next Auth เป็น 3rd Party Library สนับสนุนการทำ Authentication สำหรับ Next.js
  - นิยมใช้ในการ implement ฟังก์ชั่นการ log-in โดยใช้ credential ประเภทต่าง ๆ
  - รองรับการทำงานกับ database ต่าง ๆ
    - MySQL, MariaDB, Postgres, SQL Server, MongoDB and SQLite
  - รองรับการทำงานกับ servicer ต่าง ๆ
    - OAuth, OAuth 2.0, Open ID Connect, External API
    - Active Directory, LDAP ฯลฯ
  - บริหารจัดการ Session สำหรับ user ที่ authenticate แล้ว
    - JSON Web Tokens (JWT) พร้อมการเข้ารหัสด้วย A256GCM และ database sessions
  - ทำงานกับ Next.js ทั้งแบบ pages router และแบบ app router

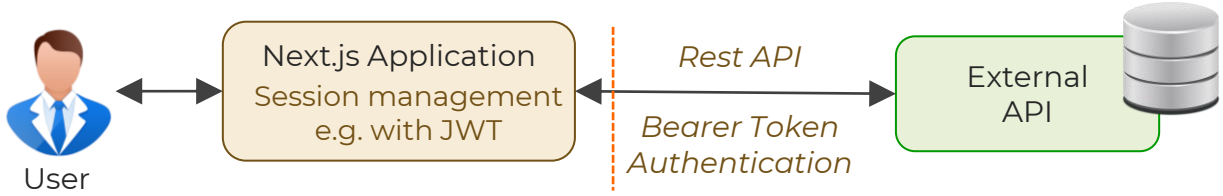


ข้อมูลเพิ่มเติม

- https://next-auth.js.org

Next Auth

- **Authentication Provider**
  - Next Auth รองรับรูปแบบการ Authenticate (Log-In Option) หลากหลาย ผ่าน provider ประเภทต่าง ๆ ตัวอย่างเช่น
    - **Credentials Provider** บริหารจัดการการ sign-in และ credentials ที่ได้รับ เช่น การใช้ Username/Password, Two-Factor Authentication
    - **Google Provider** พิสูจน์ตัวตนและบริหาร identity ผ่าน Google Account
    - **GitHub Provider** พิสูจน์ตัวตนและบริหาร identity ผ่าน GitHub Account
    - **Email Provider** พิสูจน์ตัวตนและบริหาร identity ผ่าน Email โดยการส่ง Link
    - **Keycloak Provider** พิสูจน์ตัวตนและบริหาร identity ผ่านระบบ Keycloak
  - https://next-auth.js.org/providers/
- ตัวอย่างรูปแบบการทำ Authentication ด้วย Next Auth



Next Auth Settings



- การ Setup Next Auth ต้องติดตั้งแพคเกจ version 4.24.15 ผ่าน

```
npm install next-auth@4.24.15
```

Environment Variable สำหรับ NextAuth

- Next Auth สามารถตั้งค่า option ผ่าน environment variable เช่น
  - NEXTAUTH_URL เป็น canonical URL of your site ของ website

```
NEXTAUTH_URL=https://example.com
```

  - NEXTAUTH_SECRET เป็นค่า secret ที่ใช้ในการ encrypt NextAuth JWT และใช้ในการ hash email verification tokens
    - ค่านี้จะถูกใช้เป็น default value สำหรับ `secret` option ใน NextAuth และใน Middleware

Next Auth Settings

การ Initialize NextAuth

- สำหรับ Next.js Application ที่ทำงานด้วย App Router (app/) ให้ใช้ Route Handler ในการกำหนดให้ NextAuth ดัก Request และใช้ handler ในการจัดการกับ Request
  - สร้างไฟล์ app/api/auth/[...nextauth]/route.ts บรรจุ route handler สำหรับ NextAuth
  - NextAuth ตรวจจับการถูก initialize ด้วย Router Handler ซึ่งรับ web Request และจะ return Handler (ซึ่ง Handler นี้ return web Response)
    - Initialize NextAuth ด้วยฟังก์ชั่น NextAuth() ซึ่งรับ argument เป็น NextAuth Options authOptions ซึ่ง เป็น Global Configuration สำหรับ NextAuth
    - authOptions บรรจุ Providers ที่ใช้งาน
    - NextAuth ต้องการทั้ง GET และ POST handler ในการทำงาน จึงต้อง export ทั้งคู่

app/api/auth/[...nextauth]/route.ts

```
import NextAuth from "next-auth"

const handler = NextAuth({
  ...
})

export { handler as GET, handler as POST }
```

authOptions กำหนด Global Configuration สำหรับ NextAuth

Next Auth Settings

Route Handler

- ตัวอย่างไฟล์ `route.ts` ที่ใช้งาน Credentials Provider

ส่วน `credentials` กำหนด form สำหรับหน้า sign-in

ฟังก์ชั่น `authorize` บรรจุ logic การทำ authentication และ authorization โดย default คืนค่าเป็น `user` property สำหรับ JWT ของ Next Auth

ระบุชนิด Provider ว่าเป็น `CredentialsProvider`

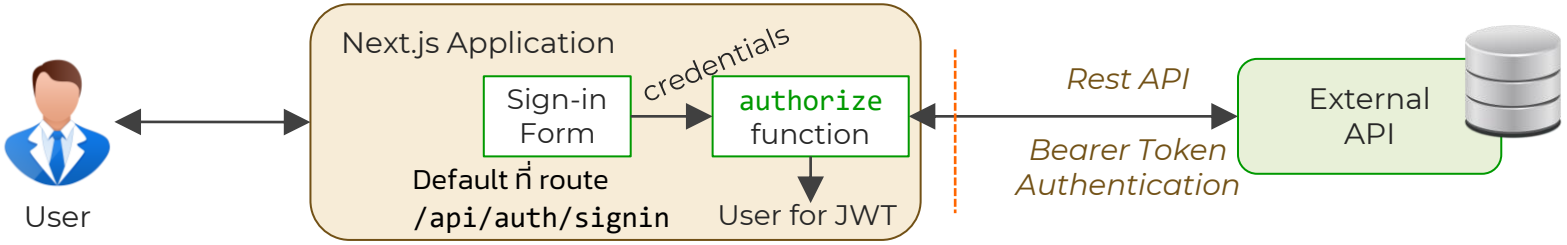
```
import CredentialsProvider from "next-auth/providers/credentials";
...
providers: [
  CredentialsProvider({
    // The name to display on the sign in form (e.g. "Sign in with...")
    name: "Credentials",
    // `credentials` is used to generate a form on the sign in page.
    // You can specify which fields should be submitted, by adding keys to the `credentials` object.
    // e.g. domain, username, password, 2FA token, etc.
    // You can pass any HTML attribute to the <input> tag through the object.
    credentials: {
      username: { label: "Username", type: "text", placeholder: "jsmith" },
      password: { label: "Password", type: "password" }
    },
    async authorize(credentials, req) {
      // Add logic here to look up the user from the credentials supplied
      const user = { id: "1", name: "J Smith", email: "jsmith@example.com" }

      if (user) {
        // Any object returned will be saved in `user` property of the JWT
        return user
      } else {
        // If you return null then an error will be displayed advising the user to check their details.
        return null

        // You can also Reject this callback with an Error thus the user will be sent to the error page
      }
    }
  })
]
...
```

Next Auth Settings

- รูปแบบการทำ Authentication ผ่าน Credential Provider



การทำงานกับ Session Object ใน Next Auth บน Server Component

- บน Server-Side เช่น Route Handlers และ React Server Components สามารถเข้าถึง session object ผ่านฟังก์ชั่น

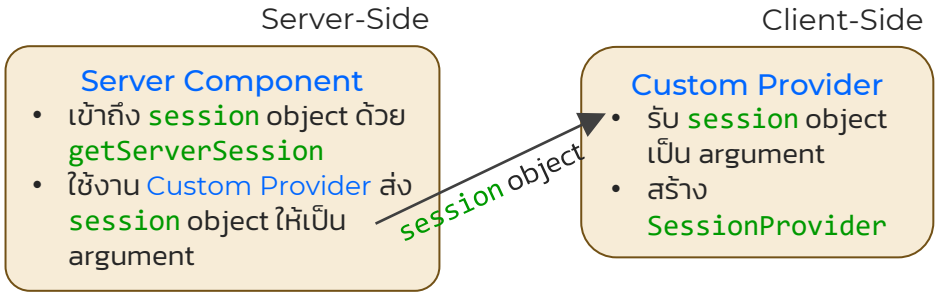
```
session = await getServerSession(authOptions)
```

- *authOptions* Global Configuration สำหรับ NextAuth ที่ใช้ตอน Initialize NextAuth
- *session* Session Object ที่ถูกสร้างจากการทำ Authentication

Next Auth Session

การทำงานกับ Session Object ใน Next Auth

- Next Auth จดจำ Authentication Session ผ่าน `SessionProvider`
  - `SessionProvider` ต้องการใช้ Session Object ซึ่งได้มาจากการเรียก `getServerSession` บน Server-Side
  - อย่างไรก็ตาม `SessionProvider` ใช้งาน React Context ซึ่งเป็นกลไกในการบริหารจัดการ state ใน React โดย React Context ทำงานบน Client-Side Component เท่านั้น ทำให้ไม่สามารถใช้งาน `SessionProvider` บน Server Component ได้โดยตรง
- แนวทางการใช้งาน `SessionProvider`
  - สร้าง Custom Provider เป็น `SessionProvider` บน Client-Side
  - Server-Side component ใช้งาน Custom Provider ส่ง Session Object ซึ่งได้มาจาก `getServerSession` มาให้ Custom Provider



Next Auth Session



การทำงานกับ Session Object ใน Next Auth บน Client Component

- Client Component เข้าถึง session object ผ่าน `useSession` hook

```
const {data: session, status} = useSession()
```

`useSession` hook return ค่าเป็น object บรรจุข้อมูล 2 ชิ้น ได้แก่

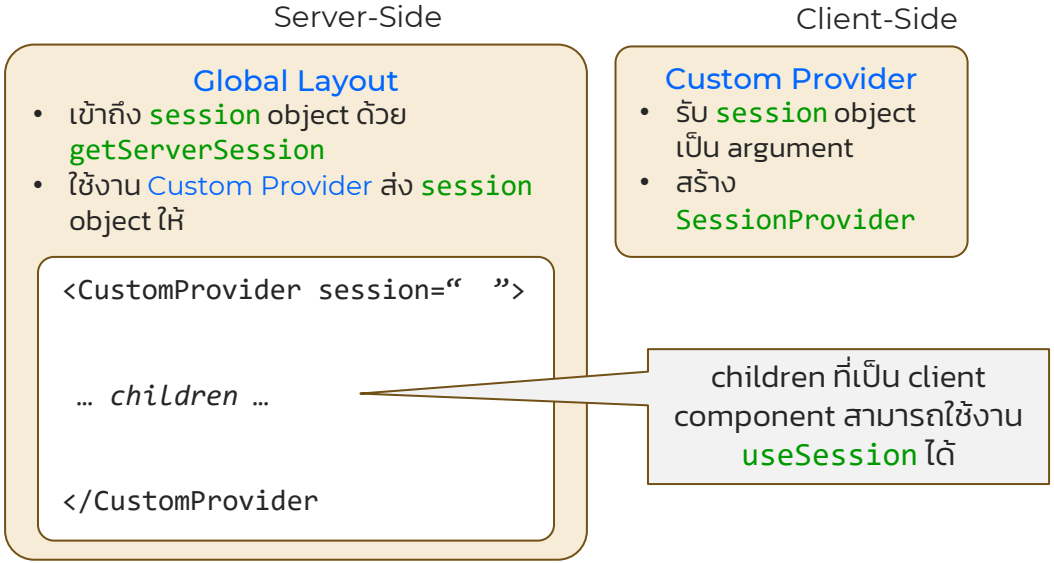
- `data` มีค่าที่เป็นไปได้ 3 ค่า ได้แก่
  - `undefined` เมื่อยังไม่มีการ fetch session
  - `session` เมื่อเข้าถึง session สำเร็จ
  - `null` เมื่อเข้าถึง session ไม่สำเร็จ
- `status` เป็น enum แสดงสถานะ ได้แก่ `loading` หรือ `authenticated` หรือ `unauthenticated`

- `useSession` hook ให้ session data และ authenticated status บน client side จึงเป็น hook สำหรับตรวจสอบข้อมูลการ sign-in
- `useSession` ต้องถูก wrap อยู่ภายใต้ `SessionProvider`

Next Auth Session

การทำงานกับ Session Object ใน Next Auth บน Client Component

- useSession ต้องถูก wrap อยู่ภายใต้ SessionProvider จึงมีแนวทางการ implement ดังนี้



Next.js Middleware

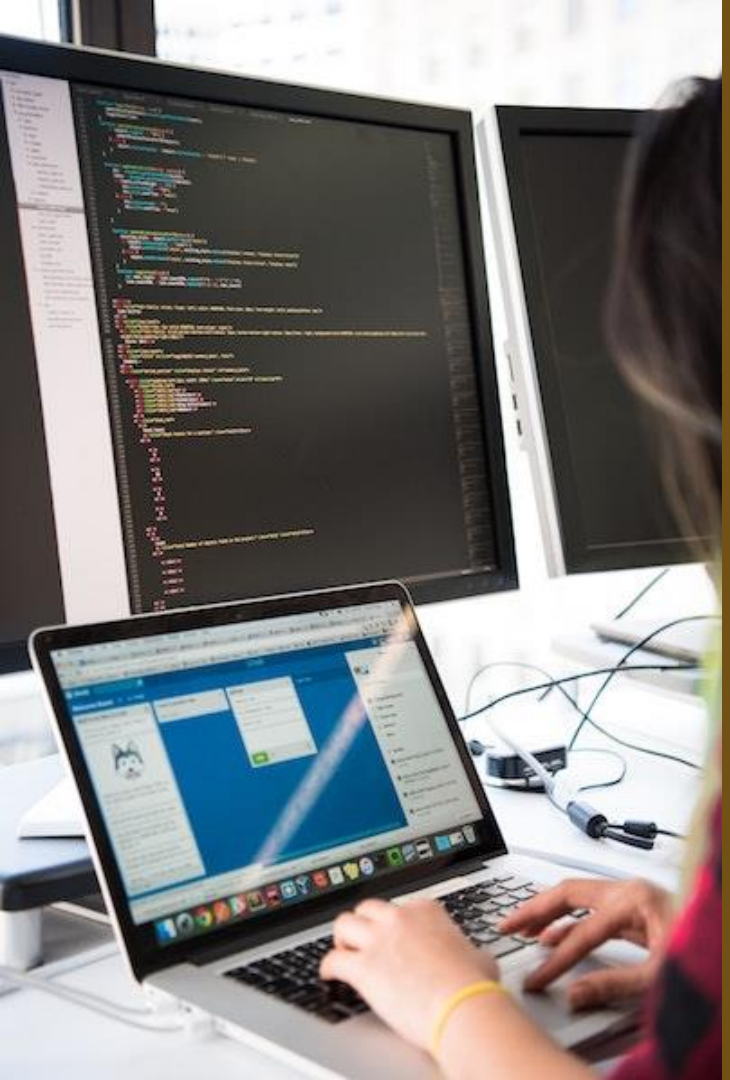
Route Protection ด้วย Middleware

- Next Auth รองรับการสร้าง protected route ในระดับ middleware
- Next.js สนับสนุน middleware ซึ่งเป็นกลไกที่ใช้ในการ run code ก่อนที่ operate request สำเร็จ
  - พิจารณา incoming request เพื่อเอื้อต่อการ modify response ด้วยการ rewrite, redirect, modify request หรือ response headers หรือทำการ respond กลับโดยตรง
  - Middleware run ก่อนการ match cached content หรือ route
- Implement middleware โดยการสร้างไฟล์ `middleware.ts` ไว้ใน project root directory
  - ตัวอย่างการ configure middleware ด้วยการใช้ matcher

middleware.js

```
export const config = {
  matcher: ['/about/:path*', '/dashboard/:path*'],
}
```

https://nextjs.org/docs/app/building-your-application/routing/middleware



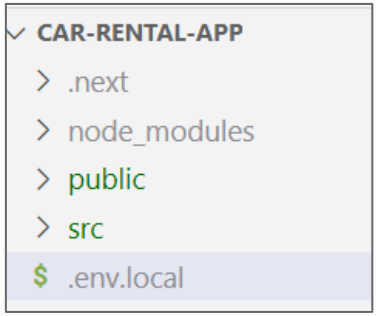
Workshop 10.1: Using Next Auth for Authentication Management

Using Next Auth

- Stop Server
- ติดตั้ง Next Auth version 4.24.15 ผ่าน console ด้วยคำสั่ง

```
npm i next-auth@4.24.15
```

- ประกาศ environmental variable สำหรับ Next Auth โดย
  - สร้างไฟล์ .env.local
  - ตั้งค่า environment variable NEXTAUTH_URL=http://localhost:3000 และ NEXTAUTH_SECRET='Ey7nTKnggBc0bRN8WUjyShw2qzOZ6KW4fUyqcKBePxY='



.env.local

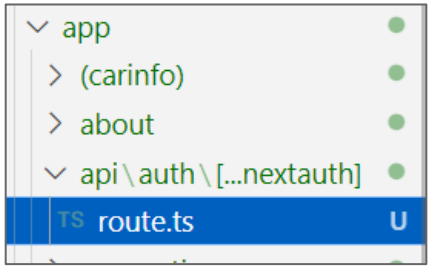
```
1  NEXTAUTH_URL=http://localhost:3000
2  NEXTAUTH_SECRET='Ey7nTKnggBc0bRN8WUjyShw2qzOZ6KW4fUyqcKBePxY='
```

เป็นค่าที่ generate จากการ random

Using Next Auth



- สร้าง API Route สำหรับจัดการ authentication request จาก Next Auth โดยสำหรับ Next.js application ที่ใช้งาน app router (ใช้ไดเร็กทอรี่ app เป็น router) สร้างโดย
  - สร้างไดเร็กทอรี่ `app/api/auth/[...nextauth]/` และสร้างไฟล์ `route.ts`
    - ดู Template จาก https://next-auth.js.org/configuration/initialization#route-handlers-app
  - เลือกใช้งาน `CredentialsProvider` เพื่อ authenticate ผ่าน username และ password




```ts
import NextAuth from "next-auth";
import { AuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";

export const authOptions:AuthOptions = {
    providers: [
        // Authentication Provider, use Credentials Provider

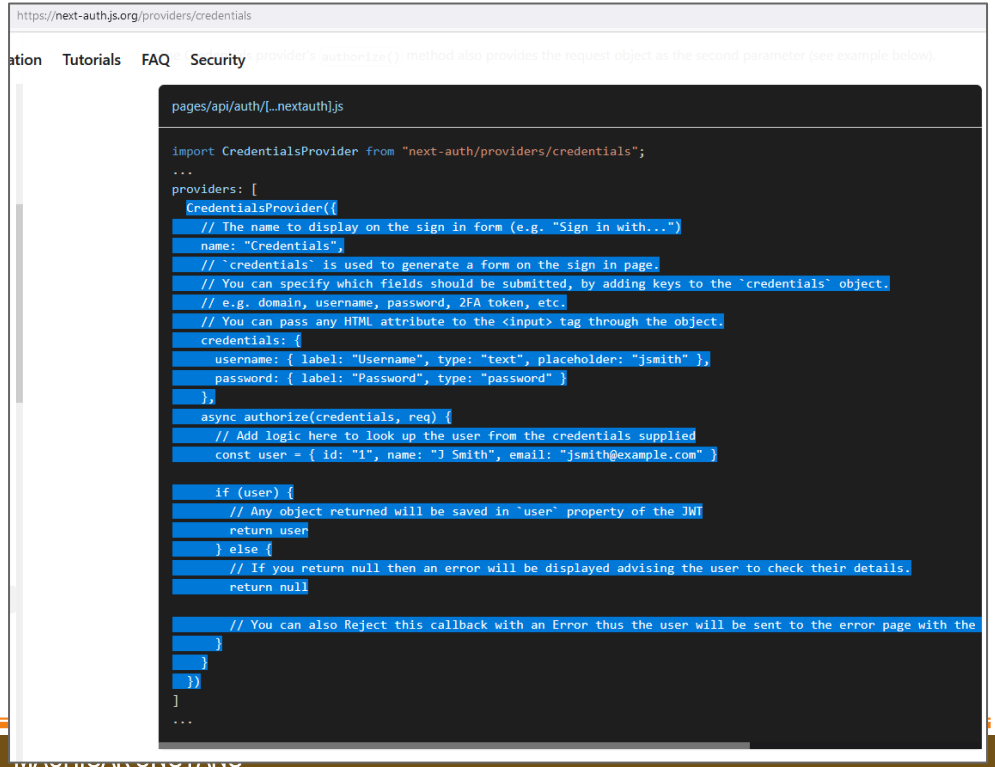
    ],
    session: { strategy: "jwt" }
}
const handler = NextAuth(authOptions)
export { handler as GET, handler as POST };
```

ใช้ `CredentialsProvider`

ใช้ `JWT` ในการจัดการ session

Using Next Auth

- Set การใช้งาน Credentials Provider ของ Next Auth
  - ใช้ template เริ่มต้น จาก https://next-auth.js.org/configuration/providers/credentials



วาง template ใน api/auth/[...nextauth]/route.ts

```
// Authentication Provider, use Credentials Provider
CredentialsProvider({
  // The name to display on the sign in form (e.g. "Sign in with...")
  name: "Credentials",
  // `credentials` is used to generate a form on the sign in page.
  // You can specify which fields should be submitted, by adding keys to the `credentia
  // e.g. domain, username, password, 2FA token, etc.
  // You can pass any HTML attribute to the <input> tag through the object.
  credentials: {
    username: { label: "Username", type: "text", placeholder: "username" },
    password: { label: "Password", type: "password" }
  },
  async authorize(credentials, req) {
    // Add logic here to look up the user from the credentials supplied
    const user = { id: "1", name: "J Smith", email: "jsmith@example.com" }

    if (user) {
      // Any object returned will be saved in `user` property of the JWT
      return user
    } else {
      // If you return null then an error will be displayed advising the user to check
      return null

      // You can also Reject this callback with an Error thus the user will be sent to
    }
  }
})
```

Using Next Auth



- **Credentials Provider Template**
  - ส่วน `credentials` กำหนด form สำหรับหน้า sign-in โดย default มี
    - Username textbox
    - Password textbox
  - ส่วนฟังก์ชั่น `authorize` บรรจุ logic การทำ authentication และ authorization โดย default
    - Return user J Smith

```
// Authentication Provider, use Credentials Provider
CredentialsProvider({
  // The name to display on the sign in form (e.g. "Sign in with...")
  name: "Credentials",
  // `credentials` is used to generate a form on the sign in page.
  // You can specify which fields should be submitted, by adding keys to the `credentia
  // e.g. domain, username, password, 2FA token, etc.
  // You can pass any HTML attribute to the <input> tag through the object.
  credentials: {
    username: { label: "Username", type: "text", placeholder: "username" },
    password: { label: "Password", type: "password" }
  },
  async authorize(credentials, req) {
    // Add logic here to look up the user from the credentials supplied
    const user = { id: "1", name: "J Smith", email: "jsmith@example.com" }

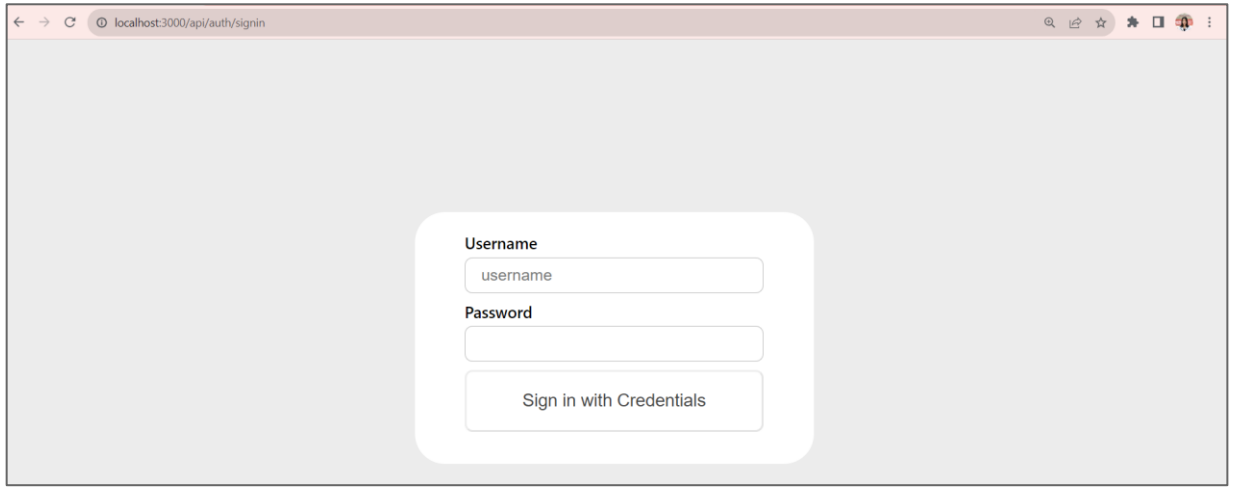
    if (user) {
      // Any object returned will be saved in `user` property of the JWT
      return user
    } else {
      // If you return null then an error will be displayed advising the user to check
      return null

      // You can also Reject this callback with an Error thus the user will be sent to
    }
  }
})
```

Using Next Auth

- เมื่อตั้งค่าการใช้งาน Next Auth แล้ว โดย default หน้า sign-in และ sign-out ของ Next Auth อยู่ที่
  - หน้า sign-in ของ Next Auth อยู่ที่ http://localhost:3000/api/auth/signin
  - หน้า sign-out ของ Next Auth อยู่ที่ http://localhost:3000/api/auth/signout

- Start Server แล้ว Navigate ไปที่ http://localhost:3000/api/auth/signin



Using Next Auth

- ลองเข้าถึง User Session จาก Server Component
  - ที่ components/TopMenu.tsx เข้าถึง user session

```
import TopMenuItem from './TopMenuItem';
import { getServerSession } from 'next-auth'
import { authOptions } from '@/app/api/auth/[...nextauth]/route';
import Link from 'next/link';

export default async function TopMenu () {

    const session = await getServerSession(authOptions)

    return (
        <div className={styles.menucontainer}>
            <Image src={'/img/logo.png'} className={styles.logoimg}
            alt='logo' width={0} height={0} sizes="100vh"/>
            <TopMenuItem title='Reservations' pageRef='/reservations'/>
            <TopMenuItem title='About' pageRef='/about'/>
            {
                session? <Link href="/api/auth/signout">
                    <div className='flex items-center absolute right-0 h-full px-2 text-cyan-600 text-sm'>
                    Sign-Out of {session.user?.name} </div></Link>
                : <Link href="/api/auth/signin">
                    <div className='flex items-center absolute right-0 h-full px-2 text-cyan-600 text-sm'>
                    Sign-In</div></Link>
            }
        </div>
    );
}
```

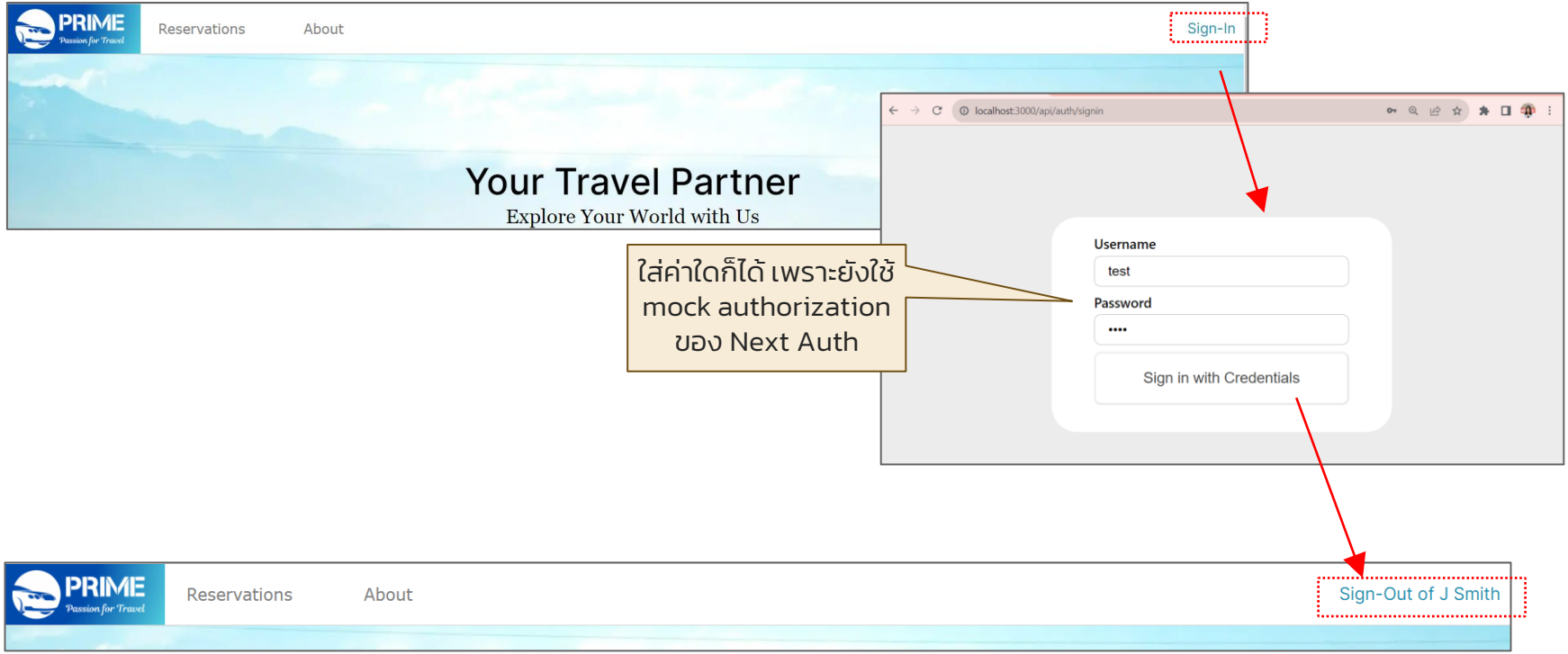
เข้าถึง Next Auth Session ผ่าน getServerSession

ถ้ามี session object ให้แสดง Link สำหรับการ Sign-Out พร้อมชื่อ user จาก credentials

ถ้าไม่มี session object ให้แสดง Link สำหรับการ Sign-In

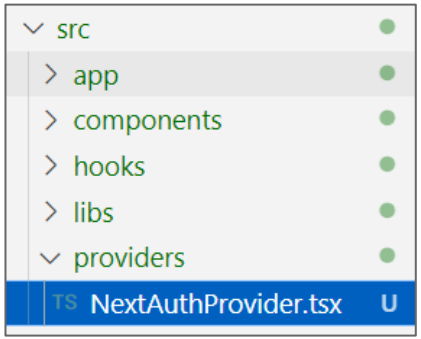
Using Next Auth

- ลองใช้งานจาก TopMenu component บนหน้าแรกของเว็บ



Next Auth

- ลองเข้าถึง User Session จาก Client Component
  - สร้าง Customized Session Provider สำหรับ Next Auth โดยให้เป็น client component
  - สร้างโฟลเดอร์ `providers` ใน `src` สำหรับเก็บ Session Provider สร้างไฟล์ `NextAuthProvider.tsx`




```tsx
"use client"
import { SessionProvider } from 'next-auth/react'

export default function NextAuthProvider({children, session}
    : {children:React.ReactNode, session:any}) : React.ReactNode {
        return (
            <SessionProvider session={session}>
                { children }
            </SessionProvider>
        )
}
```

Next Auth



- ใน Root Layout ไฟล์ app/layout.tsx ซึ่งเป็น Server Component
  - เข้าถึง Session ผ่าน getServerSession() แล้วนำไปใช้เป็น session สำหรับ NextAuthProvider
  - นำ NextAuthProvider ไปครอบ children ทั้งหมด เพื่อให้ client component ทั้งหมดเข้าถึง session ได้

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";
import TopMenu from '@/components/TopMenu'
import { getServerSession } from 'next-auth'
import { authOptions } from './api/auth/[...nextauth]/route'
import NextAuthProvider from '@/providers/NextAuthProvider'

const geistSans = Geist({
```

```
export default async function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  const session = await getServerSession(authOptions)
  return (
    <html lang="en">
```

```
const session = await getServerSession(authOptions)
return (
  <html lang="en">
    <body className={`${geistSans.variable} ${geistMono.variable}`}>
      <NextAuthProvider session={session}>
        <TopMenu/>
        {children}
      </NextAuthProvider>
    </body>
  </html>
);
```

Next Auth

- ลองเข้าถึง User Session จาก Client Component
  - ที่ components/Banner.tsx เข้าถึง user session

```tsx
import { useRouter } from 'next/navigation'
import { useSession } from 'next-auth/react'

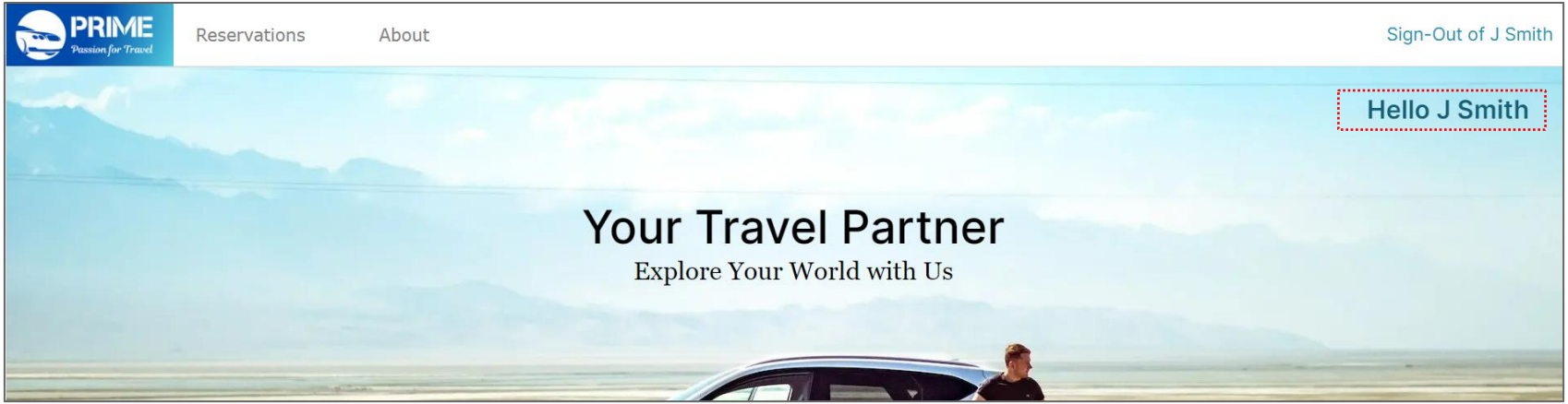
export default function Banner () {
    const covers = ['/img/cover.jpg', '/img/cover2.jpg', '/img/cover3.jpg']
    const [index, setIndex] = useState(0)
    const router = useRouter()
    const { data:session } = useSession()

    return (
        <div className={styles.banner} onClick={()=>{ setIndex(index+1) }}>
            <Image src={covers[index%3]}
            alt='cover'
            fill={true}
            objectFit='cover'/>
            <div className={styles.bannerText}>
                <h1 className='text-4xl font-medium'>Your Travel Partner</h1>
                <h3 className='text-xl font-serif'>Explore Your World with Us</h3>
            </div>
            {
                session? <div className='z-30 absolute top-5 right-10 font-semibold text-cyan-800 text-xl'>
                Hello {session.user?.name}</div>:null
            }
            <button className="bg-white text-cyan-600 border border-cyan-600
```

ถ้ามี session object ให้แสดง ชื่อ user บน Banner

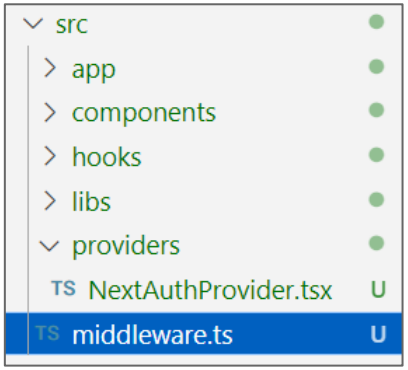
Using Next Auth

- ลองใช้งานจาก Banner component บนหน้าแรกของเว็บ



Using Next Auth

- ใน `src` สร้างไฟล์ `middleware.ts`
  - ให้ protect route `/reservations/manage`



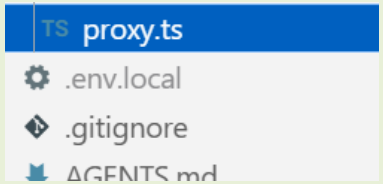

```ts
import { withAuth } from 'next-auth/middleware'
export default withAuth({})

export const config = {
    matcher: ["/reservations/manage"],
};
```

สำหรับ NextJS 16 การใช้ชื่อ `middleware.ts` นั้น deprecated

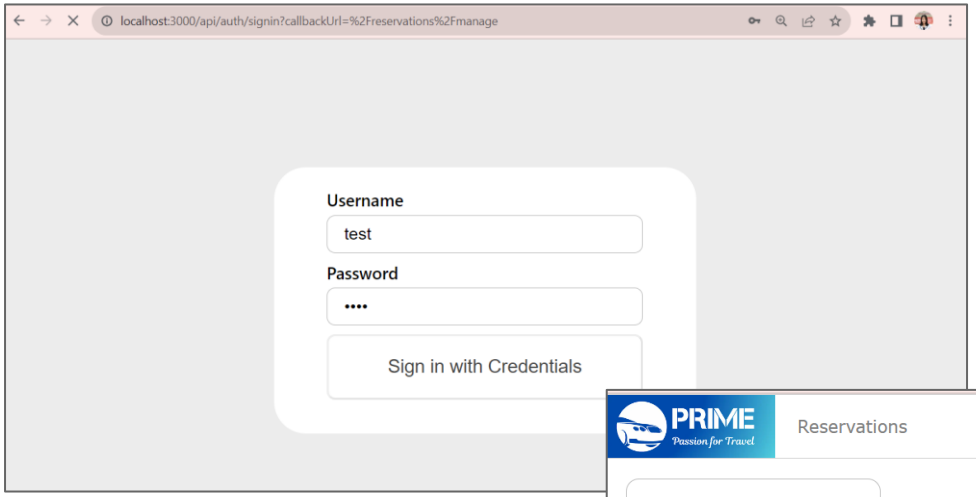
สามารถ rename middleware.ts เป็น `proxy.ts` ได้

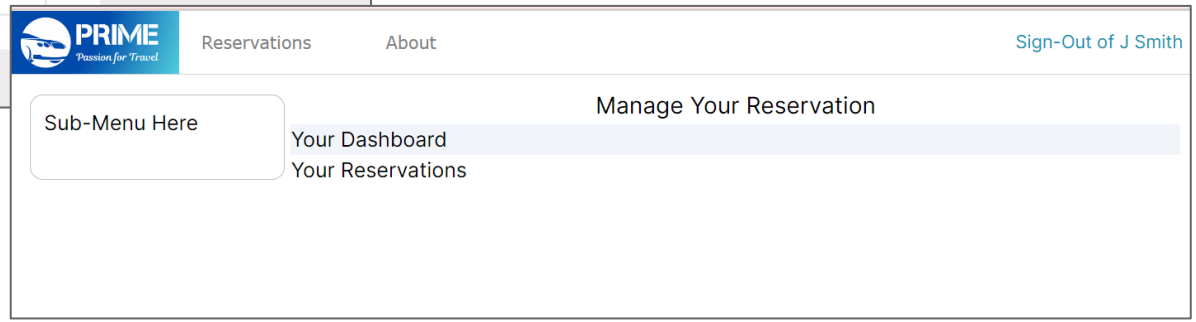
เพื่อไม่ให้มี warning



Using Next Auth

- ลอง Sign Out แล้ว navigate ไปที่ http://localhost:3000/reservations/manage
  - ระบบจะให้ sign-in แล้วจึง re-direct กลับไปที่หน้า http://localhost:3000/reservations/manage



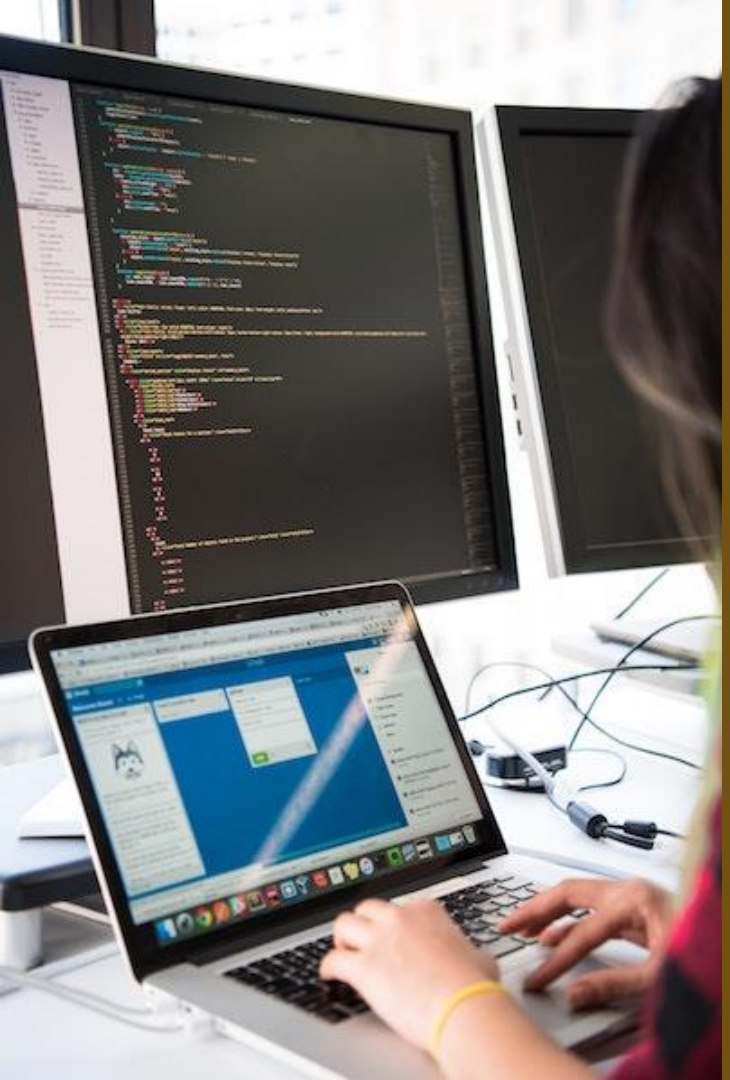


Data Posting with Fetch



- คำสั่ง `fetch` รองรับ options ในการระบุ option ต่าง ๆ ตัวอย่างเช่น
  - `method` HTTP Request Method เช่น GET หรือ POST
  - `headers` HTTP Request Header บรรจุข้อมูลเกี่ยวกับ Request เช่น
    - `Content-Type` เช่น application/json หรือ application/x-www-form-urlencoded เป็นต้น
  - `body` บรรจุ Request Body ซึ่งต้อง match กับ `Content-Type`

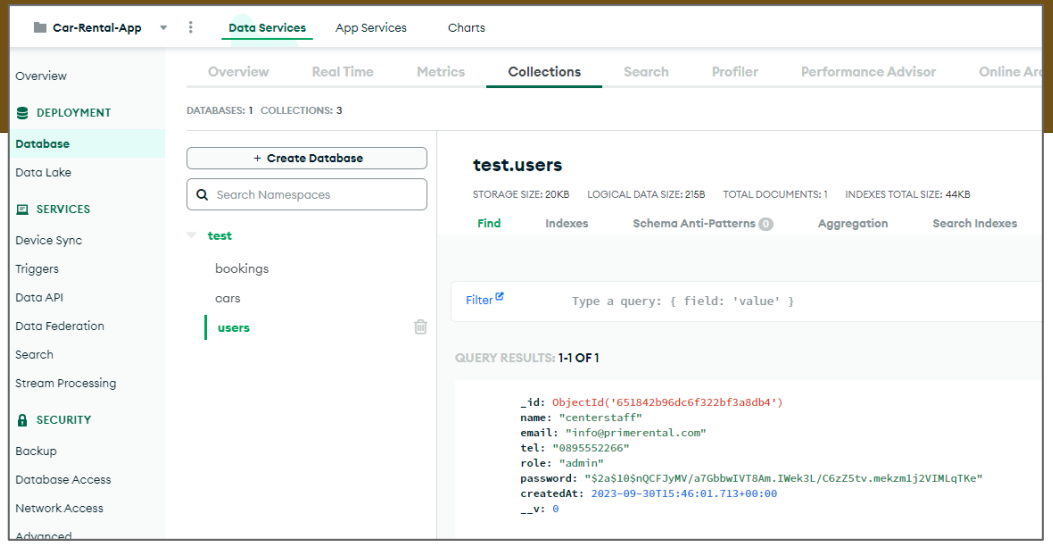
```
const response = await fetch(url, {
    method: "POST", // *GET, POST, PUT, DELETE, etc.
    mode: "cors", // no-cors, *cors, same-origin
    cache: "no-cache", // *default, no-cache, reload, force-cache, only-if-cached
    credentials: "same-origin", // include, *same-origin, omit
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(data),
  });
```



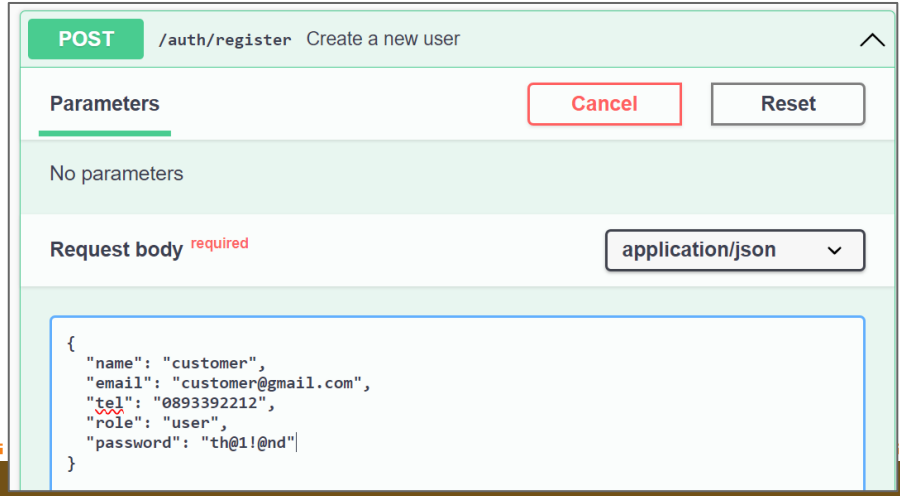
Workshop 10.2: Posting to Backend API

Using Log-In API

- เข้า MongoDB Atlas ของ Backend
  - สังเกตว่ามี admin user 1 คน ซึ่งสร้างไว้ใน Class 09



- Start Backend Application
  - ไปที่ http://localhost:5000/api-docs/
  - สร้าง user ใหม่ที่มี role เป็น user



Using Log-In API

- ลองทดสอบการ Log-In ผ่าน API Docs

Request body required

```
{
  "email": "info@primerental.com",
  "password": "$0ftw@reD3v"
}
```

- สังเกต Response ที่ได้

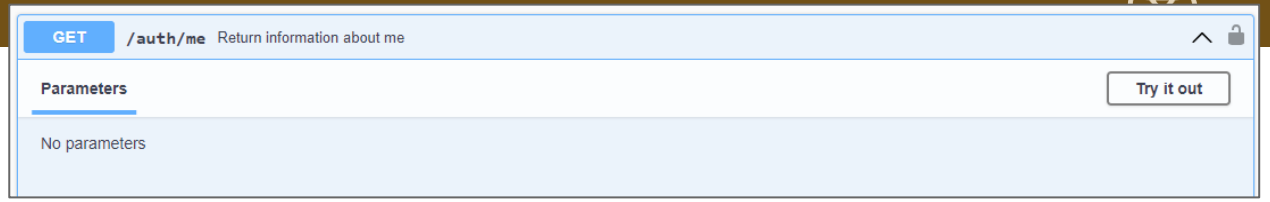
Response body


```
{
  "success": true,
  "_id": "68cf88e140ae8db8881ab29f",
  "name": "centerstaff",
  "email": "info@primerental.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4Y2Y4OGUxNDBhZThkYjg4ODFhYjI5ZiIsImlhdCI6MTc1OTU2OTcyMSwiZXhwIjoxNzkxMTA1NzIxfQ.1ejPLpLaKCwHOrE-TomNeIAhj32bPOiLK6ll
}
```

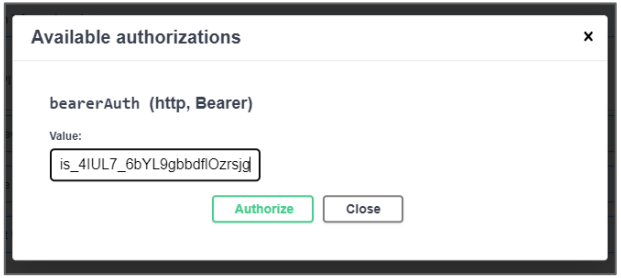

Download

Using Log-In API

- ทดสอบ API `/auth/me` สำหรับดึง user profile



- ใช้งาน token ที่ได้จากการ log-in ในการ authorize



- สังเกต Response ที่ได้

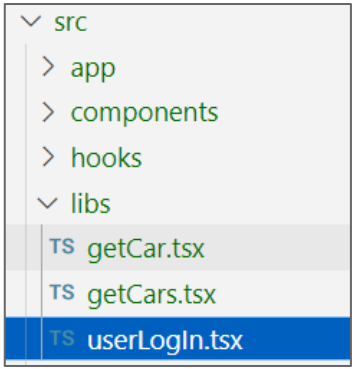
Response body


```
{
  "success": true,
  "data": {
    "_id": "68cf88e140ae8db8881ab29f",
    "name": "centerstaff",
    "email": "info@primerental.com",
    "tel": "0895552266",
    "role": "admin",
    "createdAt": "2025-09-21T05:10:57.900Z",
    "__v": 0
  }
}
```

Using Log-In API



- สร้างไฟล์ `libs/userLogin.tsx` เก็บฟังก์ชั่นการ Log-In




```tsx
export default async function userLogIn(userEmail:string, userPassword:string) {
    const response = await fetch("http://localhost:5000/api/v1/auth/login", {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
        },
        body: JSON.stringify({
            email: userEmail,
            password: userPassword,
        }),
    })

    if(!response.ok) {
        throw new Error("Failed to fetch cars")
    }
    return await response.json()
}
```

Using Log-In API



- ปรับ Credentials ใน `route.ts` ให้สอดคล้องกับ Credentials ที่ API ใช้งาน
- ให้ฟังก์ชั่น `authorize` เรียก `userLogIn` function ในการทำ authentication /authorization

```ts
import CredentialsProvider from "next-auth/providers/credentials";
import userLogIn from "@/libs/userLogIn";

export const authOptions:AuthOptions = {
```

```ts
CredentialsProvider({
  // The name to display on the sign in form (e.g. "Sign in with...")
  name: "Credentials",
  // `credentials` is used to generate a form on the sign in page.
  // You can specify which fields should be submitted, by adding keys to the `cr
  // e.g. domain, username, password, 2FA token, etc.
  // You can pass any HTML attribute to the <input> tag through the object.
  credentials: {
    email: { label: "Email", type: "email", placeholder: "email" },
    password: { label: "Password", type: "password" }
  },
  async authorize(credentials, req) {

    if(!credentials) return null

    const user = await userLogIn(credentials.email, credentials.password)

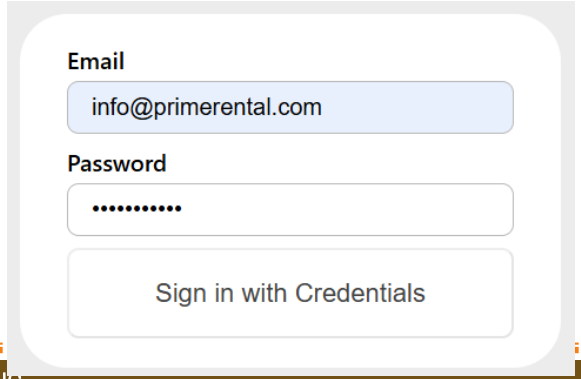
    if (user) {
      // Any object returned will be saved in `user` property of the JWT
      return user
    } else {
      // If you return null then an error will be displayed advising the user to
```

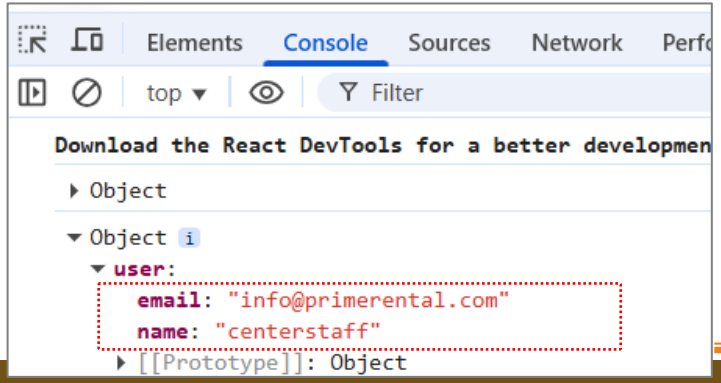
Using Log-In API

- ที่ components/Banner.tsx ให้ log ข้อมูล session ออกมาแสดงบน Browser Console

```
export default function Banner () {
    const covers = ['/img/cover.jpg', '/img/cover2.jpg', '/img/cover3.jpg']
    const [index, setIndex] = useState(0)
    const router = useRouter()
    const { data:session } = useSession()
    console.log(session)
```

- ลอง Sign-In ด้วย username และ password ที่สร้างไว้บน Backend
  - เมื่อ Sign-In แล้ว เปิด Browser Console ดูข้อมูลใน session จะพบว่ามีเฉพาะ email และ name

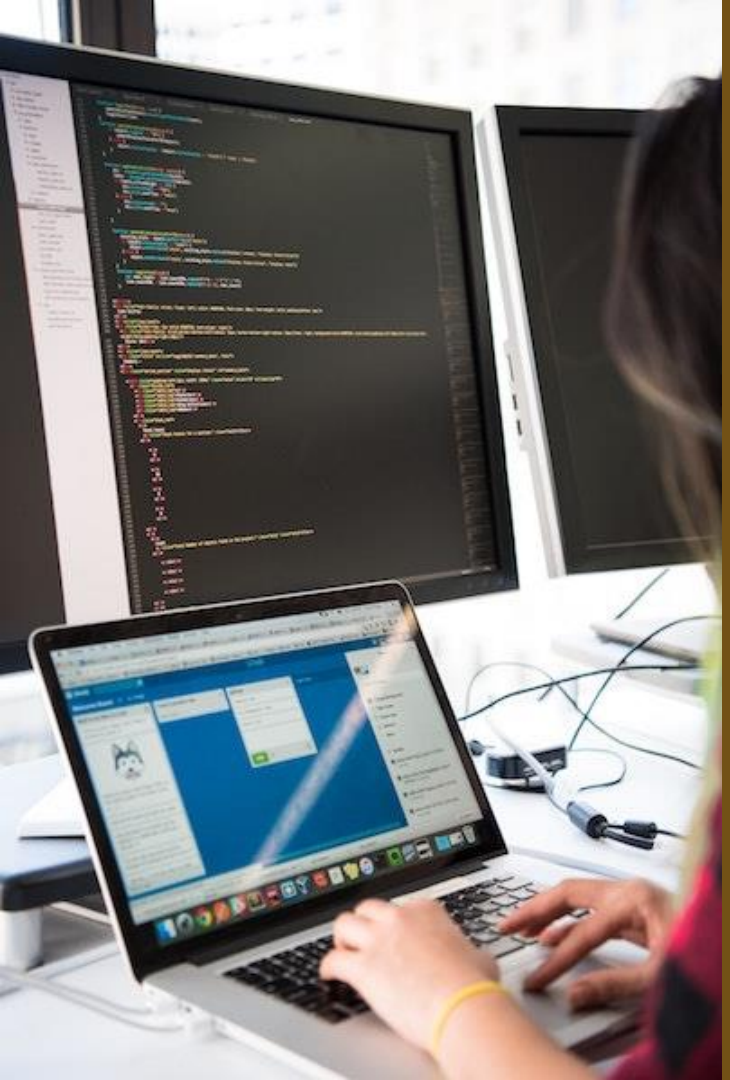




Customize NextAuth Session Object



- โดย default แล้ว NextAuth session token บรรจุเพียง basic name และ email
- Next Auth รองรับการ customize และมีกลุ่มฟังก์ชั่นที่เป็น `callbacks` สำหรับ modify content ของ session
  - `jwt()` **callback** เป็น callback ที่จะถูกเรียกเมื่อ user ทำการ log-in และ JWT Token ถูกสร้าง หรือถูก update
    - `user` object ของ JWT จะได้รับการกำหนดเนื้อหา เป็น object ที่ return มาจาก authorize function
    - `jwt()` callback คืนค่าเป็น object ที่จะเป็น JWT Token ซึ่งจะถูก save ใน session cookie
  - `session()` **callback** เป็น callback รับ session cookie content เข้ามา ผ่านพารามิเตอร์ token
    - `session()` callback คืนค่าเป็น `session` object ที่เข้าถึงได้ผ่านการเรียก `getServerSession()` หรือ `useSession` hook



Workshop 10.3: Saving Access Token

Saving Access Token



- โดย default แล้ว NextAuth session token บรรจุเพียง basic name และ email
  - แต่ข้อมูลจากการ log-in ด้วย Backend API ให้ข้อมูลมากหลายชิ้น รวมถึง token สำหรับใช้งาน API ด้วย

Response body


```
{
  "success": true,
  "_id": "65227069a916385c7d574ed5",
  "name": "customer",
  "email": "customer@gmail.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY1MjI3MDY5YTkxNjM4NWM3ZDU3NGVkNSIsImlhdCI6MTY5Njc1NTk0NCwiZXh
fQ.R-pyIg35KmRQyfU-8is_4IUL7_6bYL9gbbdflOzrsjg"
}
```


Download

- เพิ่ม data เข้าไปใน NextAuth session token โดยเพิ่ม callbacks ทั้ง `jwt()` callback และ `session()` callback เข้าไปใน `/app/api/auth/[…nextauth]/route.ts`

```
  session: { strategy: "jwt" },
  callbacks: {
    async jwt({token, user}) {

    },
    async session({session, token, user}) {

    },
  }
}
```

Saving Access Token

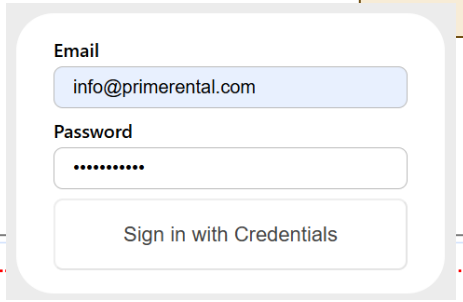
- ใน /app/api/auth/[…nextauth]/route.ts
  - ใส่รูปแบบเนื้อหาของ token และ session

```
callbacks: {
  async jwt({token, user}) {
    return {...token, ...user}
  },
  async session({session, token, user}) {
    session.user = token
    return session
  },
}
```

Spread ค่าของ NextAuth token และ Spread ค่าของ user ซึ่งได้มาจาก Backend API ออกมารวมกันเพื่อให้เป็น JWT Token ที่ save ลง session cookie

นำ JWT token จาก session cookie มากำหนดเป็นข้อมูล user ของ session

- Sign-out และ Sign-In ใหม่ จะพบว่า session token บรรจุข้อมูลจาก Backend รวมถึง token จาก Backend ด้วย




```
{user: {…}}
  user:
    email: "info@primerental.com"
    exp: 1762162737
    iat: 1759570737
    jti: "9f834e7a-d1bc-460e-bbba-8bcad55172aa"
    name: "centerstaff"
    success: true
    token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4Y2Y4OGUxNDBhZThkYjg4ODFhYjI5ZiIsImlhdCI6MTc1OTU3MDcwOCwiZXhwIjoxNzkxMTA2NzA4fQ.VbzhzijHjLhLhWRtrHwC-EbMzK08L1xH6Gd1t_uX1Wg"
    _id: "68cf88e140ae8db8881ab29f"
  [[Prototype]]: Object
```

Saving Access Token



- เพื่อให้ NextAuth รู้จัก attribute เพิ่มเติมสำหรับ user session
  - ใน `src` สร้างไฟล์ `next-auth.d.ts`

```ts
import NextAuth from 'next-auth'

declare module "next-auth" {
    interface Session {
        user: {
            _id: string,
            name: string,
            email: string,
            role: string,
            token: string
        }
    }
}
```

Declare module `next-auth` และ Customize `user` attribute ของ `Session`

ประกาศ `user` attribute ให้สอดคล้องกับที่ได้รับจาก Backend

  - ใน `/app/api/auth/[...nextauth]/route.ts` generalize data type ของ token เป็น any

```ts
    async session({session, token, user}) {
      session.user = token as any
      return session
    },
}
```

Saving Access Token



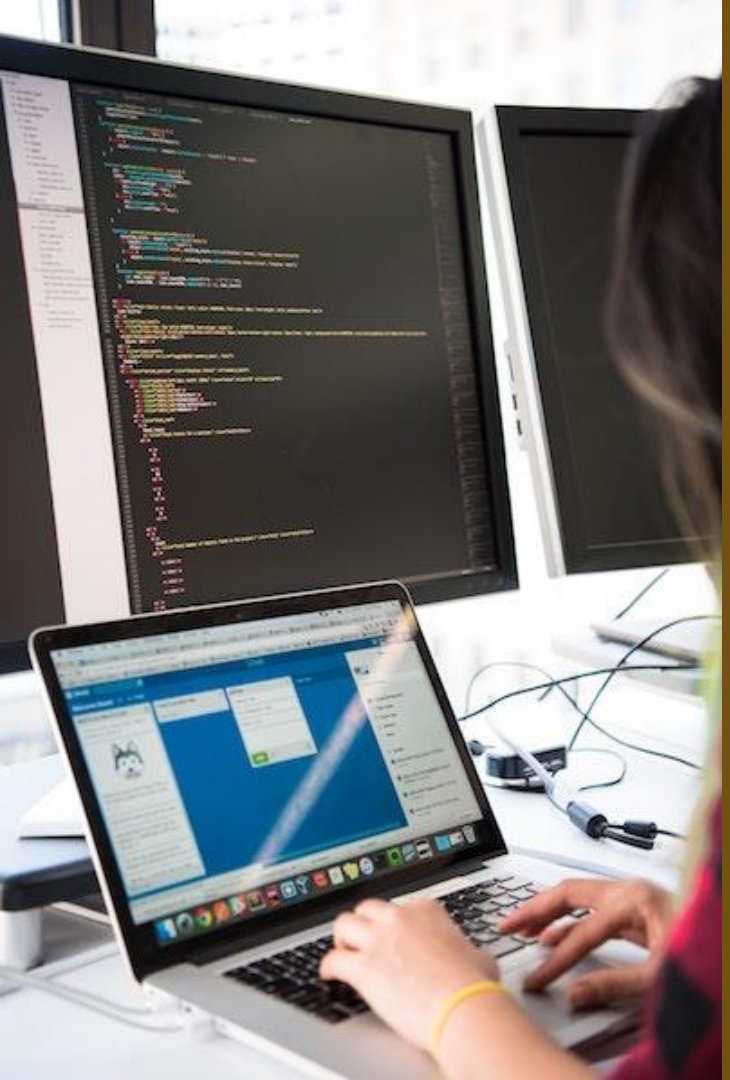
- ที่ components/Banner.tsx ให้ log ข้อมูล session token ออกมาแสดงบน Browser Console

```
export default function Banner () {
    const covers = ['/img/cover.jpg', '/img/cover2.jpg', '/img/cover3.jpg']
    const [index, setIndex] = useState(0)
    const router = useRouter()
    const { data:session } = useSession()
    console.log(session?.user.token)

    return (
```

- ทดสอบโดยเข้าไปที่หน้าแรกของเว็บ http://localhost:3000
  - สังเกต log

```
Download the React DevTools for a better development experience: https://react.dev/link/react-devtools
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4Y2Y4OGUxNDBhZThkYjg4ODFhYjI5ZiIsImlhdCI6MTc1OTU3MDcwOCwiZXhwIjoxNzkxMTA2NzA4fQ.VbzhzijHjLhLhWRtrHwC-EbMzK08LlxH6Gd1t_uX1Wg
```

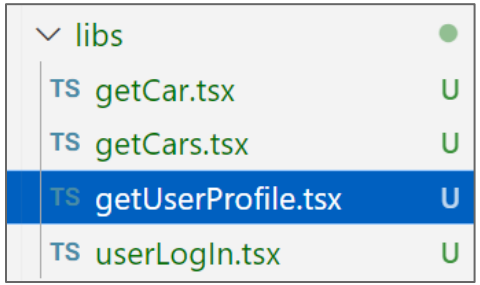


Workshop 10.4: Sending Bearer Token with Fetch

Sending Bearer Token with Fetch



- สร้างไฟล์ `libs/getUserProfile.tsx` เก็บฟังก์ชั่นการดึงข้อมูล User Profile




```tsx
export default async function getUserProfile(token:string) {

    const response = await fetch("http://localhost:5000/api/v1/auth/me", {
        method: "GET",
        headers: {
            authorization: `Bearer ${token}`,
        },
    })
    if(!response.ok) {
        throw new Error("Failed to fetch user profile")
    }
    return await response.json()
}
```

ส่ง `token` เข้าไปในส่วน `authorization` ใน `headers`

Sending Bearer Token with Fetch



- ที่หน้า app\reservations\manage\@dashboard\page.tsx
  - เข้าถึง session และ user token
  - เรียก getUserProfile() โดยส่ง User Token ไปให้
  - นำข้อมูล user profile ที่ได้มาแสดงในกล่อง dashboard

```tsx
import { authOptions } from "@/app/api/auth/[...nextauth]/route";
import { getServerSession } from 'next-auth';
import getUserProfile from "@/libs/getUserProfile";

export default async function DashboardPage () {

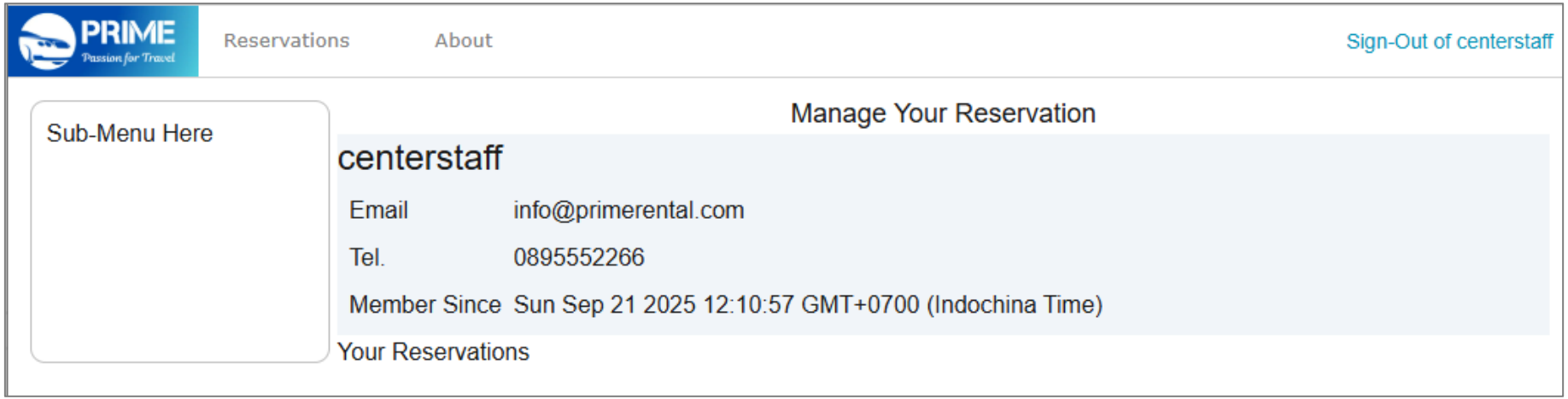
    const session = await getServerSession(authOptions)
    if(!session || !session.user.token) return null

    const profile = await getUserProfile(session.user.token)
    var createdAt = new Date(profile.data.createdAt);

    return (
        <main className="bg-slate-100 m-5 p-5">
            <div className="text-2xl">{profile.data.name}</div>
            <table className="table-auto border-separate border-spacing-2"><tbody>
                <tr><td>Email</td><td>{profile.data.email}</td></tr>
                <tr><td>Tel.</td><td>{profile.data.tel}</td></tr>
                <tr><td>Member Since</td><td>{createdAt.toString()}</td></tr>
                </tbody></table>
        </main>
    );
}
```

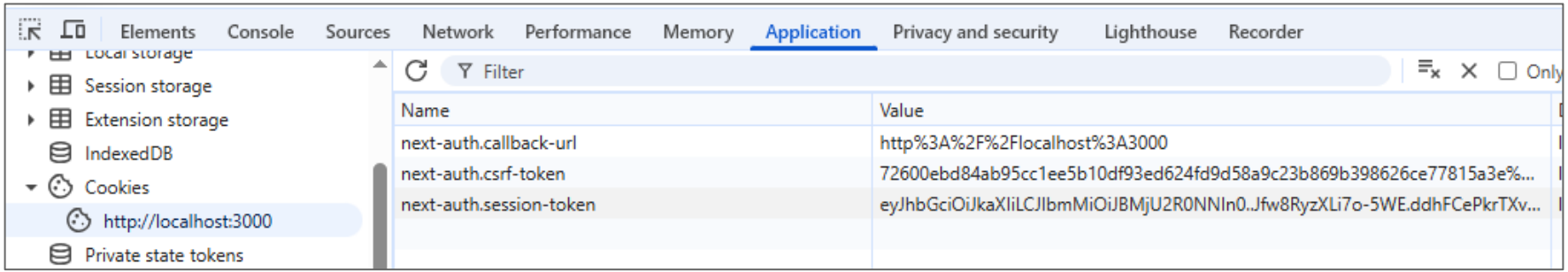
Sending Bearer Token with Fetch

- ทดสอบโดย visit route `reservations/manage`



Cookies

- Next-Auth ใช้ cookies ในการจัดเก็บข้อมูล



- การ sign out ผ่าน Next Auth ไม่ลบ client-side helper cookies ที่ไม่เกี่ยวข้องกับ session โดยตรง เช่น
  - `next-auth.callback-url` Helper cookie เก็บ Callback URL
  - `next-auth.csrf-token` Anti-CSRF

  Developer สามารถลบ cookies เหล่านี้ได้ผ่าน client-side JavaScript